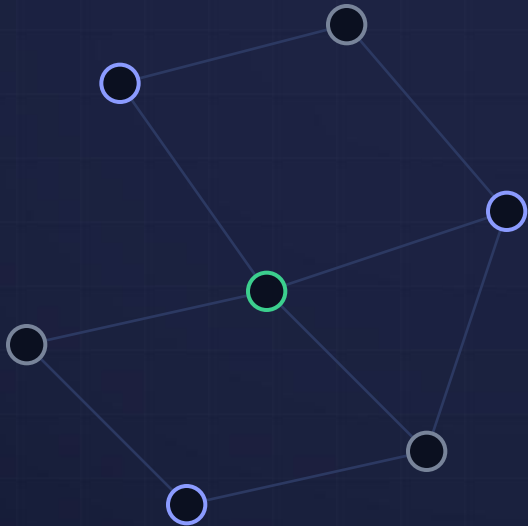


OneForAll

Repository Intelligence, Engineering Knowledge and Software Defect Risk Prediction



TEAM 14 · CSE 2023–2027 BATCH	ROLL NO.	FOCUS AREA (ZERO TH PLAN)
Sunu Aariyan	MUT23CS140	Frontend & UI
Jithin P R	MUT23CS084	Backend
Mathew Maijo Kavalam	MUT23CS103	AI & Repository Intelligence
Sahala Sajad	MUT23CS132	Database & Documentation
Individual Review-1 contributions: [PLACEHOLDER – to be confirmed by each member]		

PROJECT GUIDE

Ms. Mithu Mary George

CO-GUIDE

[PLACEHOLDER – co-guide name]

ACADEMIC YEAR

2026–27 · Semester VII

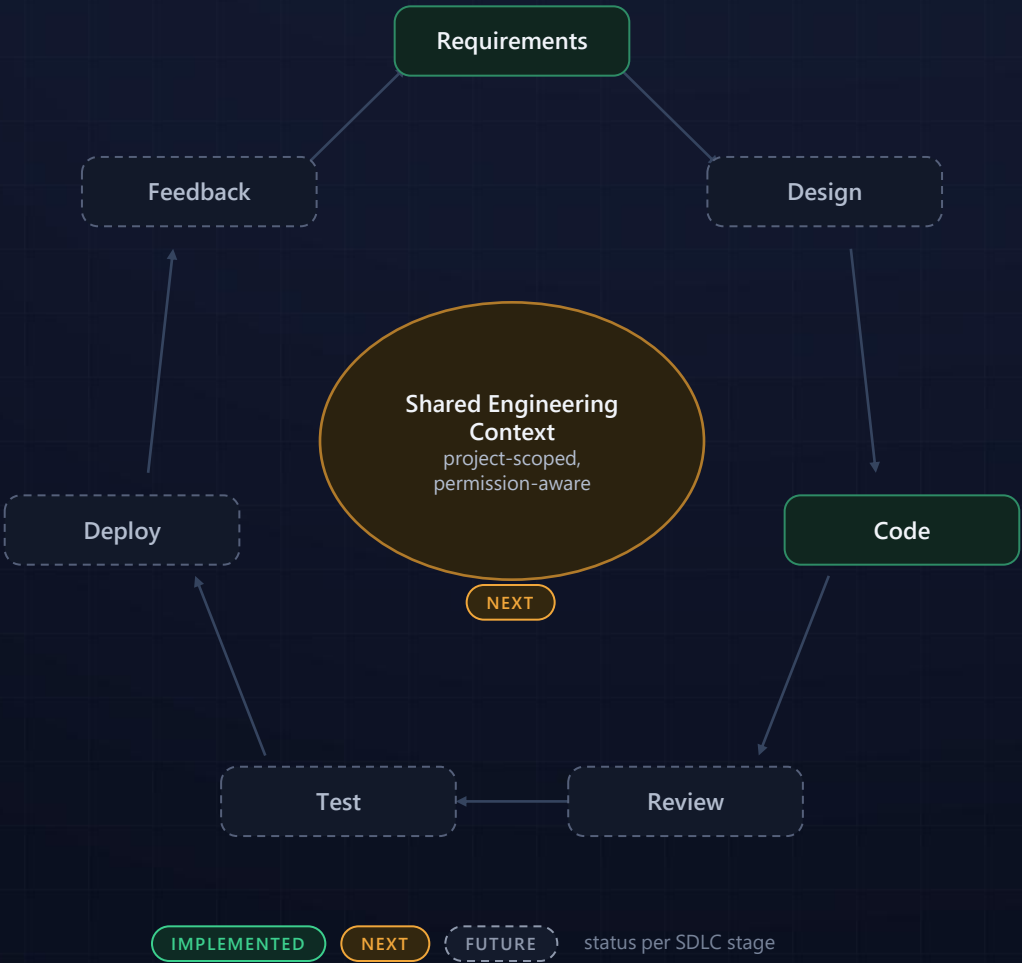
REVIEW

Project Phase I – Review 1

Project Overview

A permission-aware engineering intelligence platform, built in phases — the foundation is implemented and running.

PROBLEM	Engineering knowledge is spread across trackers, Git hosts, CI and documents; AI assistants see only fragments of a project.
MOTIVATION	Research solves single stages — code graphs, edit planning, defect prediction, explanation — but not one connected, access-controlled context.
NEED	A project-scoped engineering context that links work items with code structure, enforced by organisation roles.
TARGET USERS	Developers, team leads and project managers, QA engineers, engineering organisations.
SCOPE (PHASE I)	Multi-tenant workspace, requirements and tasks, repository snapshots and structure analysis (implemented); engineering context next.
EXPECTED IMPACT	Less manual context gathering and earlier visibility of risky changes — to be evaluated in later phases; no measured claims yet.



Progress Since Zeroth Review

What changed between the Zeroth Review (Aug 2026) and Review 1 (Oct 2026).

ASPECT	AT ZEROth REVIEW	NOW (REVIEW 1)
Problem definition	Repository-aware, multi-agent engineering intelligence platform.	Focus refined to repository intelligence, engineering knowledge and defect-risk prediction; multi-agent AI moved to later phases. Title change subject to Department approval.
Literature survey	Broad survey: repository-level AI, SE agents, code models, documentation, defect prediction.	Five core papers selected and mapped to one pipeline: Represent → Understand → Plan → Predict → Explain.
Architecture	Proposed: Angular 20, ASP.NET Core (.NET 9), SQL Server, Redis, SignalR, Azure.	Implemented: Next.js 16 + TypeScript, Supabase (PostgreSQL, Auth, Row-Level Security), Tree-sitter.
Algorithms	Proposed: AST parsing, code graph, engineering risk score.	Implemented: Tree-sitter AST extraction of symbols and relationships (4 languages). Graph, risk and explanation still planned.
Hardware	Development laptops; Azure cloud (proposed).	Development laptops + Docker (local Supabase). No special hardware; cloud deployment deferred.
Software built	Not started.	Phase 1 and 2A–2H complete: auth, organisation hierarchy, projects, requirements/tasks, repository, snapshots, structure analysis.

COMMITTEE SUGGESTIONS → ACTION TAKEN

SUGGESTION

[PLACEHOLDER – committee remark from the Zeroth Review]

ACTION TAKEN

[PLACEHOLDER – what the team changed in response]

Problem Statement

Fragmented engineering knowledge, and AI tools that see only part of it.

TODAY: KNOWLEDGE IN SEPARATE TOOLS



No shared, access-controlled view of the project. Each answer starts from zero.

PROBLEM STATEMENT

Engineering knowledge — requirements, code structure, change history and quality signals — is spread across separate tools. Developers connect these pieces by hand. General-purpose AI assistants work on pasted fragments without a persistent, permission-aware view of the project, and defect-prone changes are usually found late: in review, testing or production.

WHO IS AFFECTED

Developers, team leads and project managers, QA engineers, organisations adopting AI-assisted engineering.

WHY IT MATTERS

Time goes into gathering context; broad AI access to repositories risks exposing code to people who should not see it; late defects mean rework.

ENGINEERING SIGNIFICANCE

Needs structural code analysis (ASTs), multi-tenant authorization inside the database, and data models linking work items to code.

SDGS ADDRESSED

SDG 8 – Decent Work & Economic Growth · SDG 9 – Industry, Innovation & Infrastructure

Existing Systems and Their Limitations

Each category holds one slice of engineering knowledge.

CATEGORY	EXAMPLES	WHAT IT DOES WELL	LIMITATION FOR THIS PROBLEM
IDEs and code assistants	VS Code, IntelliJ IDEA, GitHub Copilot	Editing, completion, local navigation.	Context is mostly the open workspace and session; not tied to requirements or to organisation-level access rules.
Repository hosting	GitHub, GitLab, Bitbucket	Version control, pull requests, code search.	Code-centric; requirement-to-code traceability depends on manual linking conventions; no structural risk signal.
Issue trackers	Jira, Azure Boards	Planning, workflows, reporting.	Track work items but do not understand the code that implements them.
General AI assistants	ChatGPT, Claude, Gemini	Reasoning, explanation, drafting.	Each conversation starts from pasted fragments; no persistent, access-controlled project context.
Repository-level AI research	RepoGraph [1], CGM [2], CodePlan [3], SWE-agent [8]	Code graphs, planning, issue resolution on benchmarks.	Evaluated as standalone methods; no multi-tenant access control or requirement context.
Defect-prediction research	JIT-BiCC [4], FoX [5], Kamei et al. [12]	Commit-level risk, formal explanations.	Standalone models; not part of a team's day-to-day engineering workspace.

Inference: no existing tool maintains a project-scoped, access-controlled context that links work items, code structure and change risk.

Literature Review — Repository Understanding

Three core papers on representing, understanding and changing whole repositories.

[1] REPRESENT RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph Ouyang et al. · ICLR 2025	CORE IDEA Builds a line-level repository graph with tree-sitter: definition and reference nodes joined by invoke and contain edges. Retrieves k-hop ego-graphs around search terms.	CONTRIBUTION / FINDINGS Plug-in module for procedural and agent frameworks; improved all four tested methods on SWE-bench and generalised to CrossCodeEval.	LIMITATION / GAP Authors note: evaluated only with GPT-4-series models, on SWE-bench Lite (cost), and on Python. Gap for OneForAll: no access control or work-item context.
[2] UNDERSTAND Code Graph Model (CGM): A Graph-Integrated LLM for Repository-Level SE Tasks Tao et al. · NeurIPS 2025	CORE IDEA Puts a repository code graph (7 node types, 5 edge types) inside an open LLM: node text encoded by CodeT5+ through an adapter; a graph-aware attention mask limits attention to neighbouring nodes.	CONTRIBUTION / FINDINGS Agentless Graph-RAG pipeline (rewriter, retriever, reranker, reader); reports 43.00% resolved on SWE-bench Lite with Qwen2.5-72B, first among open-weight models.	LIMITATION / GAP Authors note: repositories larger than the context length and complex dynamic dependencies remain hard. Supports Python and Java.
[3] PLAN CodePlan: Repository-level Coding using LLMs and Planning Bairi et al. · Proc. ACM Softw. Eng. (FSE) 2024	CORE IDEA Frames repository-level coding as planning: incremental dependency analysis and change may-impact analysis drive an adaptive plan — a chain of LLM edits.	CONTRIBUTION / FINDINGS Evaluated on C# package migration and Python temporal edits needing changes to 2–97 files; closer to ground truth than the baselines.	LIMITATION / GAP Gap for OneForAll: targets edit propagation; no requirement context, defect-risk signal or multi-tenant authorization.

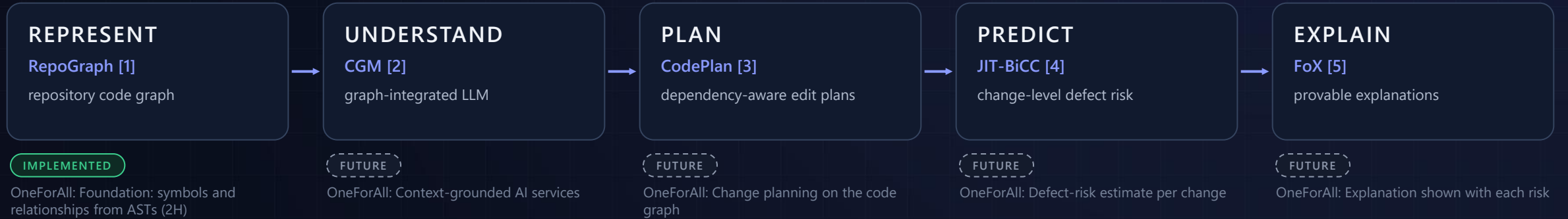
Literature Review — Defect Risk and Explanation

Two core papers on predicting and explaining risky changes, plus supporting work.

[4] PREDICT Just-In-Time Software Defect Prediction via Bi-modal Change Representation Learning Jiang, Shen, Gu · J. Syst. Softw. 219, 2025	CORE IDEA BiCC-BERT is pre-trained on code changes and commit messages with a Replaced Message Identification objective, learning bi-modal change representations.	CONTRIBUTION / FINDINGS JIT-BiCC fuses these representations with 14 expert change metrics in an MLP classifier; reports a 10.8% F1 improvement over eight baselines on 27,391 changes.	LIMITATION / GAP Gap for OneForAll: predicts per-commit risk only; explaining individual predictions and project context are outside its focus.
[5] EXPLAIN A Formal Explainer for Just-In-Time Defect Predictions (FoX) Yu, Fu, Ignatiev, Tantithamthavorn, Stuckey · ACM TOSEM 33(7), 2024	CORE IDEA Uses formal reasoning about a JIT model's behaviour to give provably correct explanations of why a commit is predicted defective, guaranteed to be minimal.	CONTRIBUTION / FINDINGS Explanations shown to be correct, robust and actionable where model-agnostic techniques are not; 54-practitioner survey: 86% found it useful, 74% trustworthy.	LIMITATION / GAP Gap for OneForAll: explains an existing JIT model; it is not a predictor and is not integrated into a team's workflow.
SUPPORTING REFERENCES (FROM THE ZEROth REVIEW SURVEY) [6] RepoCoder — iterative retrieval and generation for repository-level code completion (EMNLP 2023). [7] SWE-bench — benchmark of real GitHub issues for repository-level issue resolution (ICLR 2024). [8] SWE-agent — agent–computer interface for LLM agents working on repositories (NeurIPS 2024). [9] CodeBERT — bimodal pre-trained model for programming and natural language (Findings of EMNLP 2020). [10] GraphCodeBERT — pre-training code representations with data flow (ICLR 2021). [11] Zeng et al. — empirical study of deep just-in-time defect prediction versus simpler baselines (ISSTA 2021). [12] Kamei et al. — large-scale study of just-in-time quality assurance and change-level metrics (IEEE TSE 2013).			

Literature Insight and Research Gap

The five papers form one pipeline. Each solves one stage in isolation.



RESEARCH GAP

- **Isolated stages.** Each method is evaluated on its own benchmark (SWE-bench, JIT datasets); none keeps a persistent context for a real team's project.
- **Missing links.** Requirements and tasks are absent from repository-level methods; code structure is absent from JIT defect prediction.
- **No access boundary.** Tenant isolation and role checks before context is used are outside their scope.

NOVELTY OF ONEFORALL

- **Engineering context (Phase 2I, planned).** Requirements, tasks and code structure linked per project and kept across snapshots.
- **Authorization first.** Access is enforced in the database (RLS) before any context is read — the same boundary future AI services must pass.
- **Integration, not reinvention.** One pipeline carries the same context from representation to explained risk; published techniques are adopted and evaluated, not re-invented.

Therefore, there is a need to develop a permission-aware platform that maintains project-scoped engineering context from repositories and work items, and uses it for repository understanding and explainable defect-risk prediction.

Objectives and Expected Outcomes

Six measurable objectives; status as of Review 1.

OBJECTIVE		MEASURE OF SUCCESS	
		IMPLEMENTEDNEXTFUTURE	
O1	Secure multi-tenant workspace Organisation → department → team → project, with OWNER / ADMIN / MEMBER roles enforced in the database.	Cross-tenant and role tests pass (153/153 PostgREST checks).	IMPLEMENTED
O2	Engineering work capture Requirements and tasks per project, with lifecycle rules enforced by database triggers.	Invalid transitions rejected; regression suites pass.	IMPLEMENTED
O3	Repository intelligence foundation Snapshots, file manifests and AST extraction of symbols and relationships for TypeScript, JavaScript, Python and Java.	Parser tests 166/166; signed, tamper-checked persistence.	IMPLEMENTED
O4	Shared engineering context Link requirements and tasks to code entities per project (Phase 2I).	Every work item linkable to code; links respect RLS.	NEXT
O5	Context-grounded AI assistance Authorization-aware orchestrator that assembles only permitted context for AI services.	Answers cite only context the user may access.	FUTURE
O6	Explainable defect-risk prediction Change-level risk estimate with an explanation, informed by JIT-BiCC [4] and FoX [5].	Precision / recall / F1 against published JIT baselines.	FUTURE
EXPECTED DELIVERABLES Web prototype (running, partial) · Repository intelligence module · Engineering context store (next) · Risk prediction and explanation module (planned) · Project report · Conference paper (planned)			

Scope of the Project

What is built, what comes next, and what is planned — clearly separated.

IMPLEMENTED

Phase 1, 2A–2H

- Email / password authentication, sessions, protected routes
- Organisations, departments, teams and memberships
- Roles: OWNER, ADMIN, MEMBER
- Projects (greenfield / brownfield) with lifecycle
- Requirements and tasks with status rules
- Repository connection record (no cloning, no credentials stored)
- Snapshots and file manifests (path, language, size, SHA-256)
- Structure analysis: symbols and relationships, 4 languages
- Enterprise web UI (desktop, tablet, mobile)

NEXT

Phase 2I and prerequisites

- Shared Engineering Context: link requirements and tasks to code entities
- Repository source ingestion (fetch file content for analysis)
- Job-based execution and recovery for long analyses

Starts after Review 1. Source ingestion and jobs are prerequisites for analysing real repositories.

FUTURE

Later phases

- AI Orchestrator: intent, authorization check, context assembly, model routing
- AI services: Q&A, review, test generation, documentation
- Code graph traversal and change history
- Defect-risk prediction with explanations
- CI/CD integration and deployment feedback

Planned for the rest of Phase I and Phase II; order may change after Phase 2I.

OUT OF SCOPE

Replacing the IDE, Git host, issue tracker or CI system · Autonomous production changes · Unrestricted AI access to repositories

Functional and Non-Functional Requirements

Functional requirements by subsystem; non-functional requirements as design constraints.

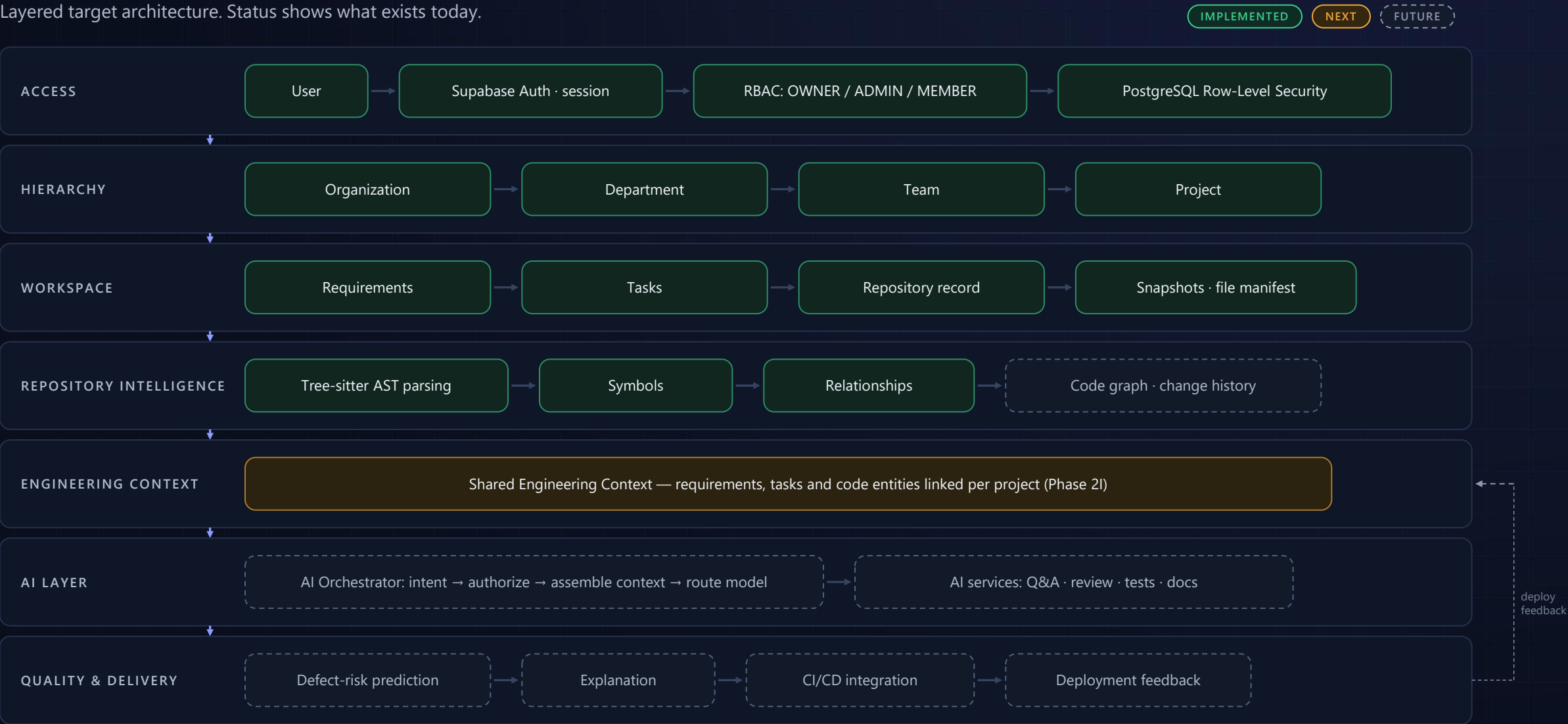
FUNCTIONAL REQUIREMENTS		
Identity and access	Sign up, sign in, sign out; session refresh; route protection.	IMPLEMENTED
Organisation hierarchy	Create organisations, departments, teams; manage memberships and roles.	IMPLEMENTED
Projects	Greenfield / brownfield projects; lifecycle DRAFT → ACTIVE → ARCHIVED.	IMPLEMENTED
Requirements and tasks	Create, prioritise and move through enforced status transitions.	IMPLEMENTED
Repository	Record one repository per project (GitHub, GitLab, Bitbucket, generic Git); disconnect.	IMPLEMENTED
Snapshots	Record commit snapshots and file manifests.	IMPLEMENTED
Structure intelligence	Extract symbols and relationships per snapshot; show file coverage.	IMPLEMENTED
Engineering context	Link work items to code entities.	NEXT
AI, risk, delivery	AI assistance; defect risk with explanation; CI/CD feedback.	FUTURE

NON-FUNCTIONAL REQUIREMENTS

- **Security.** Authorization enforced in the database (RLS) and re-checked by the application; strict tenant isolation.
- **Integrity.** Generated structure is written only through a signed server path; completed analyses are immutable.
- **Reliability.** Explicit lifecycle states; failures recorded with a retry path; no partial results stored.
- **Maintainability.** Strict TypeScript, modular server code, Zod validation, versioned SQL migrations.
- **Usability.** One design system; responsive layouts; visible keyboard focus.
- **Bounded work.** 1 MB per source file; 50,000 symbols per analysis. Performance not yet benchmarked.
- **Privacy.** No repository credentials stored; secrets stay server-side.

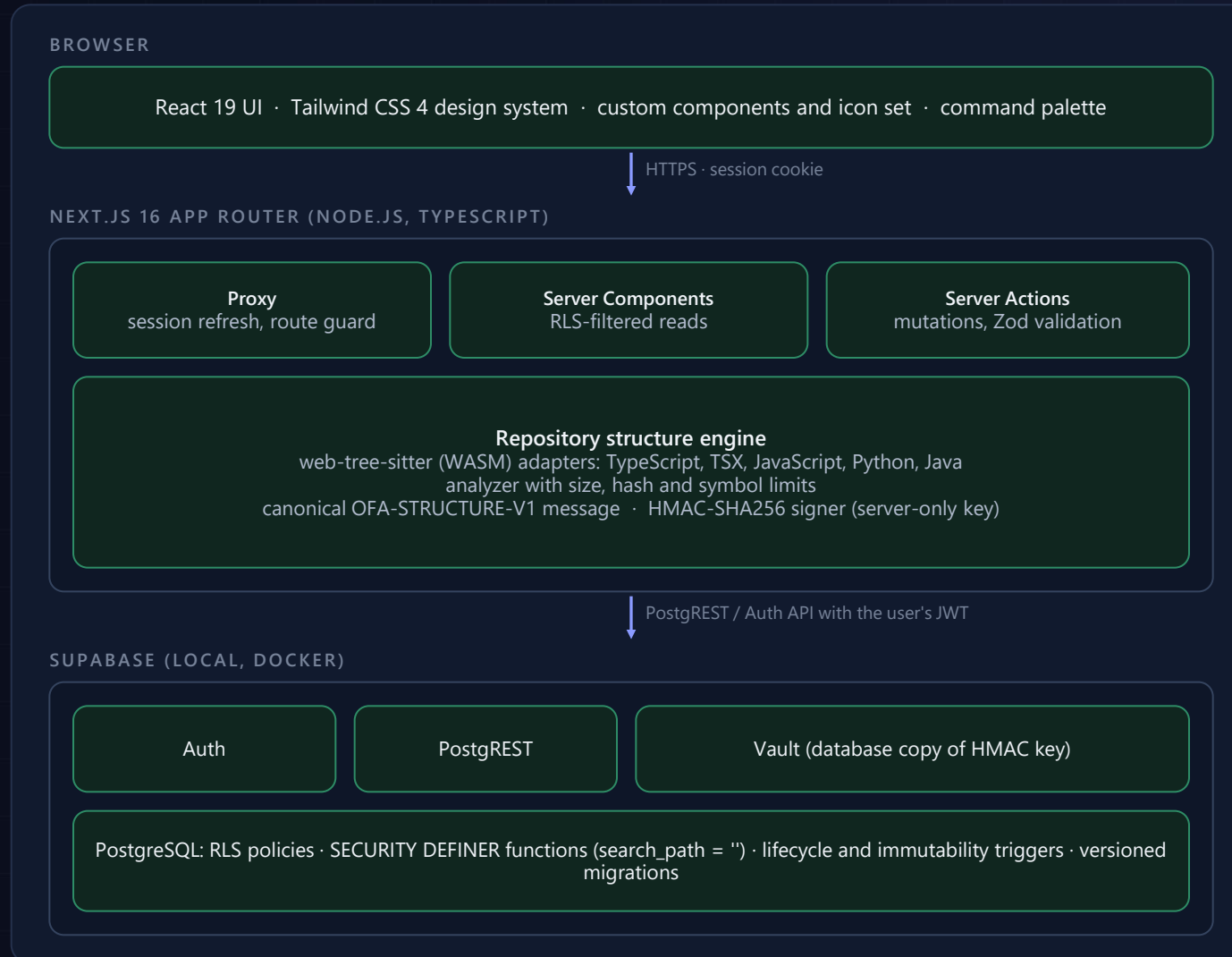
System Architecture

Layered target architecture. Status shows what exists today.



Current Implementation Architecture

Only technology that is in the codebase today.



WHY THESE TECHNOLOGIES

- **Next.js + TypeScript.** One typed language for UI and server; Server Actions keep every mutation on the server.
- **Supabase / PostgreSQL.** Auth plus Row-Level Security enforce tenant isolation inside the database, not only in app code [14], [15].
- **Tree-sitter.** Robust parsers with official grammars for many languages; runs as WASM inside Node.js [13].
- **Zod.** Runtime validation of every input before it reaches the database.
- **Tailwind CSS 4.** Token-based design system shared by every screen.

NOT IN THE CURRENT BUILD

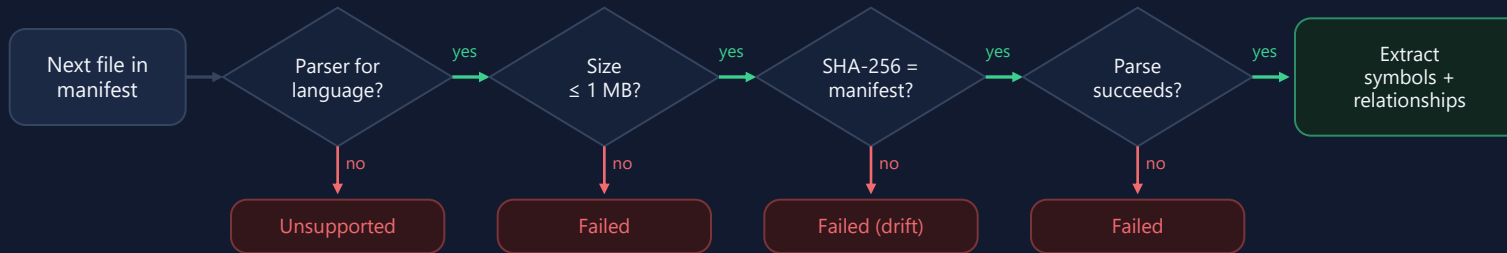
No LLM calls, embeddings, vector store, RAG pipeline or job queue yet. These belong to later phases and are not claimed here.

Repository Intelligence — Implementation

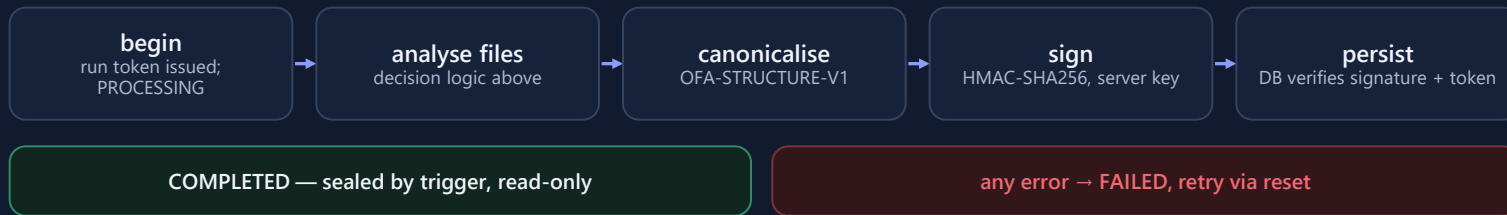
From a recorded repository to signed, queryable code structure. No AI, embeddings or RAG in this phase.



PER-FILE DECISION LOGIC (ANALYZER)



ANALYSIS LIFECYCLE (DATABASE-ENFORCED)



Limits: 1 MB per file, 50,000 symbols per analysis. Unsupported languages are counted, not guessed. Source content is checked against the manifest hash before parsing, so a changed file cannot be analysed as the recorded one.

DEMO RUN (REAL OUTPUT)

10-file snapshot · 4 languages

42

symbols

42

relationships

4 / 10

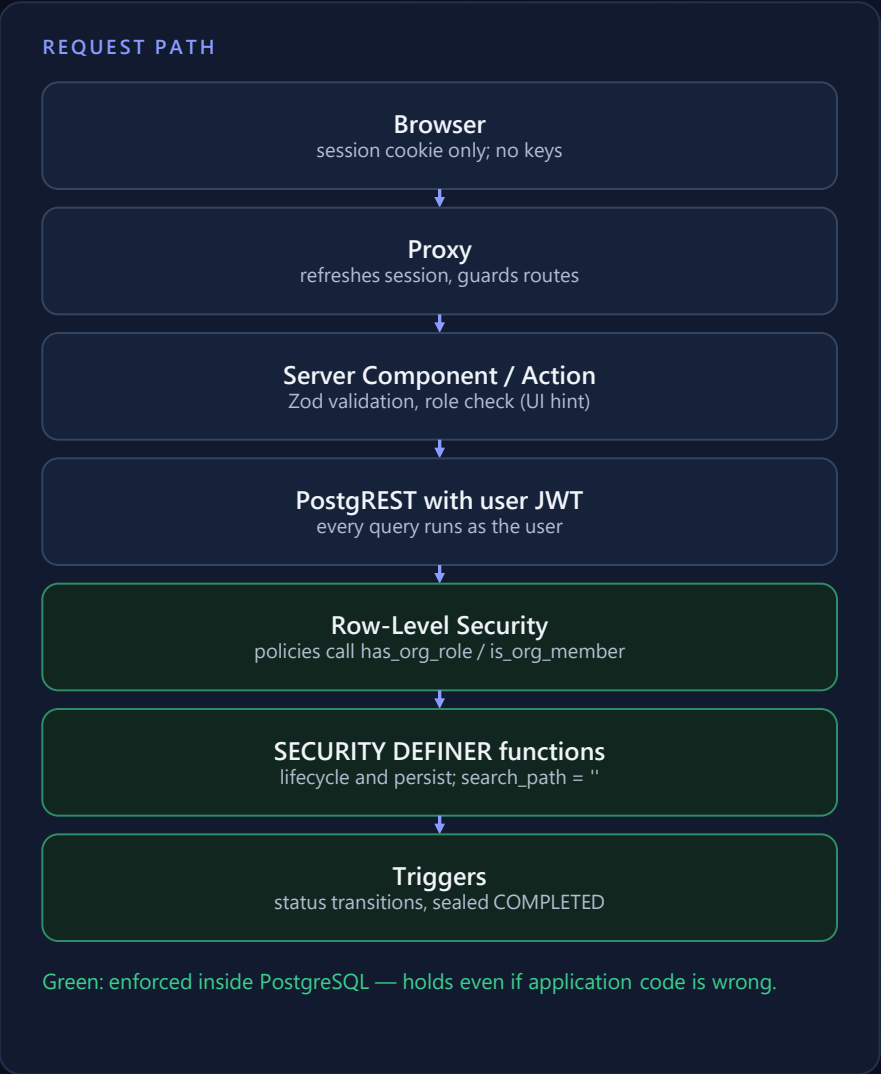
files analysed

- **Unsupported 3:** Markdown, CSV, no-language file
- **Failed 3:** hash drift, over 1 MB, syntax error
- **Relationships:** 33 contains, 5 imports, 3 extends, 1 implements
- **Symbol kinds:** 9 methods, 9 modules, 7 classes, 4 functions, 4 variables, 3 constants, 2 interfaces, 2 constructors, 1 enum, 1 type

Source content for this run came from an in-memory test provider. Production source ingestion is not built yet; the app shows a readiness message instead of fake analysis.

Security and Authorization

Authorization lives in the database; the application re-checks it but is never the only guard.



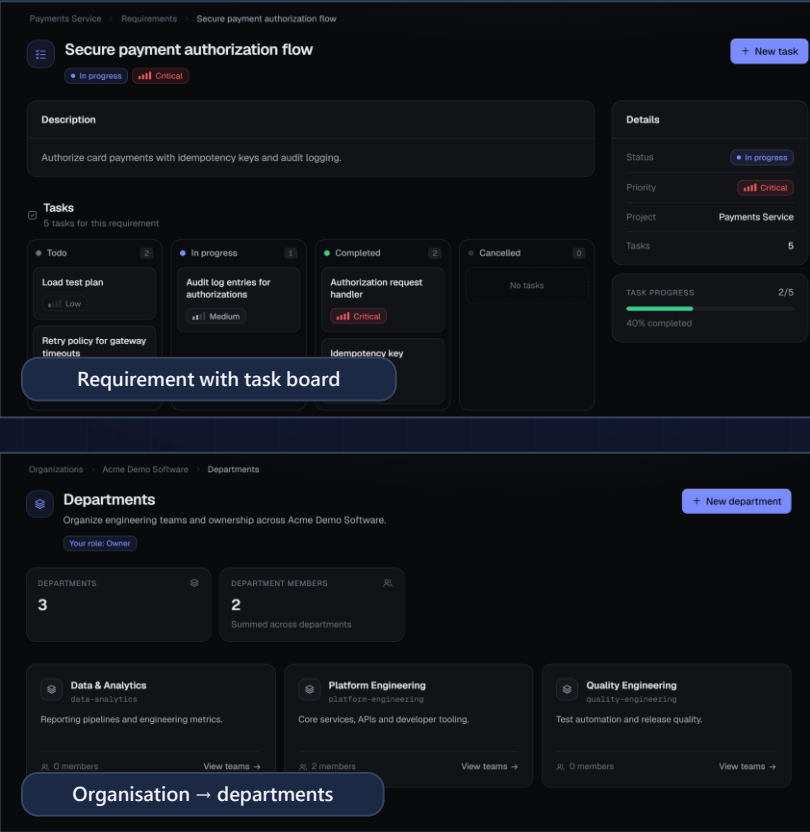
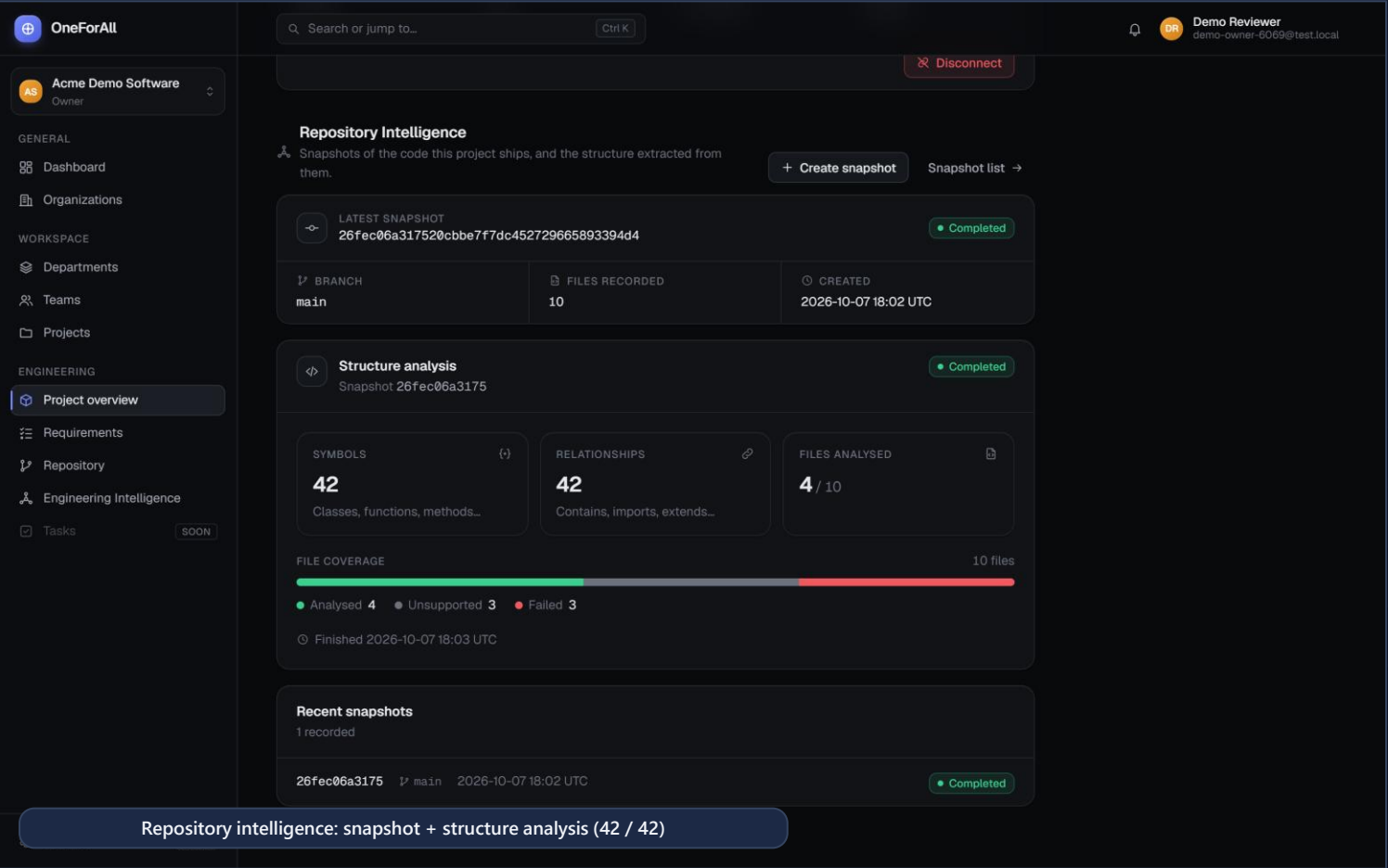
CAPABILITY	OWNER	ADMIN	MEMBER
View organisation, departments, teams	✓	✓	✓
Edit organisation; manage departments, teams, memberships	✓	✓	–
Delete organisation	✓	–	–
Create projects, requirements, tasks; repository, snapshots, analyses	✓	✓	–
View projects and their data	all	all	own teams

KEY CONTROLS

- **Cross-tenant isolation.** Another organisation's IDs return 404 — no existence leak.
- **Signed persistence.** Structure is written only by a definer function that verifies an HMAC-SHA256 signature and a one-time run token.
- **Key handling.** Server key in an environment variable, database copy in Supabase Vault; never sent to the client or logged.
- **Least privilege.** Clients cannot insert or delete symbols and relationships, or update analyses, directly.

Working Prototype — Real Screenshots

Running locally on Supabase with a fictional demo organisation (Acme Demo Software) and @test.local accounts.



- **Implemented screens:** sign-in, dashboard, organisation / department / team / project pages, requirements and task board, repository, snapshots, structure intelligence.
- **Labelled honestly:** Tasks and Settings show “Soon”. The analysis shown is real analyzer output on 10 fixture files supplied by a test provider (slide 14).

Testing and Validation

Automated checks run against the local Supabase stack. No AI or performance figures — those modules are not built.

155

mutants generated

Mutation testing (Phase 2H)

151 killed · 4 proven equivalent · 0 unexplained survivors

153 / 153

PostgREST checks pass

Phase 2H persistence over HTTP

authorization, write-path and tamper checks via the real API

166 / 166

parser tests pass

Tree-sitter adapters

TypeScript, TSX, JavaScript, Python, Java

114 / 114

manifest tests pass

Snapshot file manifests

paths, languages, sizes, hashes, isolation

BUILD AND REGRESSION CHECKS

TypeScript type check

PASS

ESLint

PASS

Production build

PASS

Database schema diff vs migrations

CLEAN

Secret scan of source and client bundle

CLEAN

Phase 2A–2G regression suites

PASS

FUNCTIONAL RESULT

Demo snapshot: 42 symbols and 42 relationships extracted from 4 of 10 files; 3 unsupported, 3 failed by design (drift, size, syntax). In verification runs the extracted symbols were identical between development and production builds.

HOW TO READ THE MUTATION SCORE

Each of the 4 equivalent mutants was shown to behave exactly like the original code, so no test can kill it. They are excluded from the score, not counted as killed: 151 of 151 non-equivalent mutants were killed. They are not failures.

OneForAll · Project Phase I – Review 1 · Team 14 · Dept. of CSE, MITS

17 / 20

Challenges and Mitigation

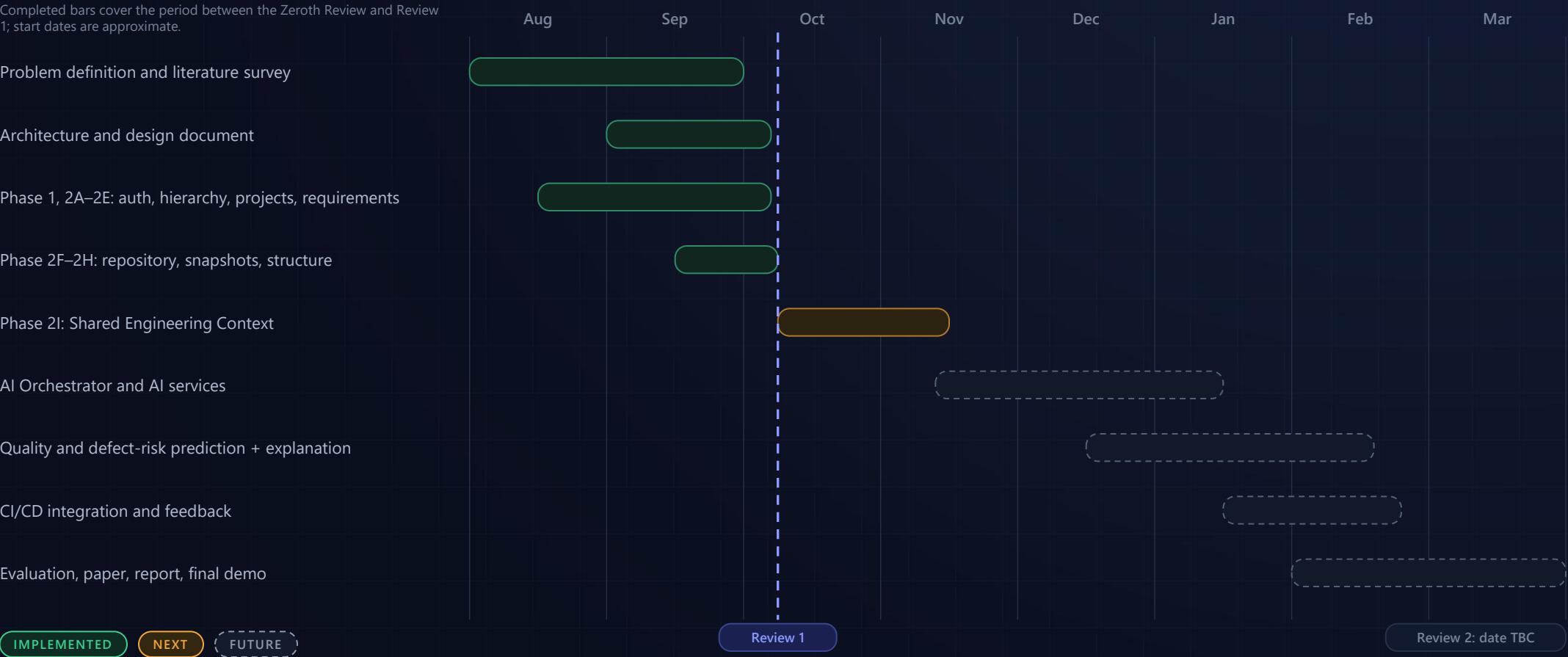
Real issues met during Phases 1 – 2H and how they were resolved.

CHALLENGE	IMPACT	MITIGATION
RLS recursion and authorization complexity	Policies on membership tables that query the same tables can recurse; role logic grew complex.	SECURITY DEFINER helper functions (has_org_role, is_org_member) with a fixed search_path; policies call the helpers.
Clients could alter generated structure	Symbols and relationships written from a client could be forged.	Client writes revoked; a definer function persists only payloads with a valid HMAC-SHA256 signature and run token; key in env + Vault.
Server Action .bind() rendering issue	Binding an action inside client-side rendering broke the page.	Bind in the Server Component and pass the bound action to the client form.
No repository source ingestion yet	Analysis cannot read real repository files in production.	Explicit readiness boundary: the UI says ingestion is required; no simulated results are ever shown.
Crash could leave an analysis in PROCESSING	A stuck analysis blocks reruns of that snapshot.	Lifecycle functions record failure and allow reset / retry; a durable job system is planned.

Updated Timeline and Remaining Work

Project schedule Aug 2026 – Mar 2027 (as in the Zeroth plan), updated with actual progress.

Completed bars cover the period between the Zeroth Review and Review 1; start dates are approximate.



REMAINING WORK

9 of 14 planned implementation phases are complete (a count of phases, not of effort). Remaining: 2I engineering context, AI orchestrator, AI services, quality / risk, CI/CD — plus evaluation and the paper.

Estimated overall completion: [PLACEHOLDER – team to confirm %]

Conclusion and References

- **Implemented and validated:** a secure multi-tenant engineering workspace and the repository-intelligence foundation (snapshots, manifests, AST-based symbols and relationships), checked by automated authorization, parser and mutation tests.
- **Research position:** five core papers form one pipeline — Represent, Understand, Plan, Predict, Explain. OneForAll's contribution is the permission-aware engineering context that connects them.
- **Next:** Phase 2I engineering context, then AI orchestration and explainable defect-risk prediction.

[1] S. Ouyang, W. Yu, K. Ma, Z. Xiao, Z. Zhang, M. Jia, J. Han, H. Zhang, and D. Yu, "RepoGraph: Enhancing AI software engineering with repository-level code graph," in Proc. ICLR, 2025.

[2] H. Tao et al., "Code Graph Model (CGM): A graph-integrated large language model for repository-level software engineering tasks," in Proc. NeurIPS, vol. 38, 2025.

[3] R. Bairi et al., "CodePlan: Repository-level coding using LLMs and planning," Proc. ACM Softw. Eng., vol. 1, no. FSE, pp. 675–698, 2024.

[4] Y. Jiang, B. Shen, and X. Gu, "Just-in-time software defect prediction via bi-modal change representation learning," J. Syst. Softw., vol. 219, Art. 112253, 2025.

[5] J. Yu, M. Fu, A. Ignatiev, C. Tantithamthavorn, and P. Stuckey, "A formal explainer for just-in-time defect predictions," ACM Trans. Softw. Eng. Methodol., vol. 33, no. 7, pp. 1–31, 2024.

[6] F. Zhang et al., "RepoCoder: Repository-level code completion through iterative retrieval and generation," in Proc. EMNLP, 2023.

[7] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?," in Proc. ICLR, 2024.

[8] J. Yang et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," in Proc. NeurIPS, 2024.

[9] Z. Feng et al., "CodeBERT: A pre-trained model for programming and natural languages," in Findings of EMNLP, 2020.

[10] D. Guo et al., "GraphCodeBERT: Pre-training code representations with data flow," in Proc. ICLR, 2021.

[11] Z. Zeng et al., "Deep just-in-time defect prediction: How far are we?," in Proc. ISSTA, 2021.

[12] Y. Kamei et al., "A large-scale empirical study of just-in-time quality assurance," IEEE Trans. Softw. Eng., vol. 39, no. 6, pp. 757–773, 2013.

[13] Tree-sitter, "Tree-sitter: An incremental parsing system," documentation. [Online]. Available: <https://tree-sitter.github.io>

[14] PostgreSQL Global Development Group, "Row Security Policies," PostgreSQL Documentation. [Online]. Available: <https://www.postgresql.org/docs/>

[15] Supabase, "Row Level Security" and "Vault," Supabase Documentation. [Online]. Available: <https://supabase.com/docs>

Thank you · Questions